

Claude Code mit OpenCode Go

Ein praktisches Backend-Tutorial fuer `claude-o-go`

`claude-o-go`

June 18, 2026

Contents

1	Ziel	3
2	Was vorhanden ist	3
3	<code>claude-o-go</code> von ueberall starten	3
3.1	Einmalige Einrichtung	3
3.2	Warum das funktioniert	4
3.3	Optionale Shell-Konfiguration	4
3.4	Taegliche Nutzung	4
4	Warum ein Proxy noetig ist	5
5	Exkurs: Localhost, Ports und Prozesse	5
6	Exkurs: HTTP-Proxy vs Backend	6
7	Modell-Kompatibilitaet	6
8	Request Flow	6
9	Exkurs: Streaming, SSE und Backpressure	7
10	Exkurs: Haeufige Netzwerkfehler	8
11	Wichtige Dateien	8
11.1	<code>claude-o-go</code>	8
11.2	<code>opencode-go-claude-proxy.mjs</code>	9

12 Lokale Checks 9

13 Verifikation 9

14 Haeufige Probleme 10

14.1 Proxy-Logs erscheinen in Claude Codes TUI 10

14.2 Selected model issue 10

14.3 Doppeltes /v1/v1/messages 10

14.4 Claude Code zeigt sonnet 10

15 Mental Model 10

1 Ziel

Dieses Tutorial erklart den Aufbau dieses Repositories und warum er funktioniert. Ziel ist, die Claude Code CLI weiter zu verwenden, aber Modell Anfragen an OpenCode Go statt an Anthropic Claude zu senden.

Claude Code bleibt der Harness: Terminal UI, Tool Calling, Dateioperationen, Permissions, Prompt-Verarbeitung und Agent Loop. OpenCode Go ist das Backend, das die eigentliche Modelanfrage erhaelt.

2 Was vorhanden ist

Dieses Verzeichnis enthaelt:

- `.env`: dein OpenCode Go API Key.
- `claude-o-go`: das Launcher-Skript, das du statt `claude` startest.
- `opencode-go-claude-proxy.mjs`: der lokale Proxy zwischen Claude Code und OpenCode Go.
- `opencode-go-claude-example.mjs`: ein direkter API-Test ohne Claude Code.

Der normale Start:

```
cd /home/user/Desktop/opencode-go-w-claude
./claude-o-go
```

Ein einzelner Prompt:

```
./claude-o-go -p "Reply with exactly: OK"
```

3 claude-o-go von ueberall starten

Du musst nicht jedes Mal in dieses Repository wechseln und es nicht in jedes Projekt kopieren. Installiere es einmal und verwende `claude-o-go` aus jedem Projektverzeichnis.

3.1 Einmalige Einrichtung

Launcher ausfuehrbar machen, falls noetig:

```
chmod +x /home/user/Desktop/opencode-go-w-claude/claude-o-go
```

Symlink in einem Verzeichnis auf deinem PATH anlegen:

```
mkdir -p ~/.local/bin
ln -s /home/user/Desktop/opencode-go-w-claude/claude-o-go \
    ~/.local/bin/claude-o-go
```

Sicherstellen, dass `~/local/bin` auf dem PATH liegt:

```
export PATH="$HOME/.local/bin:$PATH"
```

Shell neu laden:

```
source ~/.bashrc    # oder: source ~/.zshrc
```

3.2 Warum das funktioniert

Der Launcher loest seinen eigenen Symlink mit `readlink -f` auf. Dadurch findet er das echte Installationsverzeichnis, in dem `.env` und `opencode-go-claude-proxy.mjs` liegen. Claude Code selbst startet wieder in dem Verzeichnis, aus dem du `claude-o-go` aufgerufen hast, sodass dein aktuelles Projekt das Arbeitsverzeichnis bleibt.

3.3 Optionale Shell-Konfiguration

Wenn du den Key nicht in `.env` halten willst, exportiere ihn einmal in deinem Shell-Profil:

```
export OPENCODE_GO_API_KEY="sk-..."
export OPENCODE_GO_MODEL="qwen3.7-plus"
```

Der Launcher nimmt `OPENCODE_GO_API_KEY` aus der Umgebung und ueberspringt dann `.env`. Das Modell ist nicht mehr im Launcher hartcodiert; setze `OPENCODE_GO_MODEL` im Shell-Profil, damit es ueberall gilt.

3.4 Taegliche Nutzung

Aus einem beliebigen Projekt:

```
cd /path/to/any/project
claude-o-go
```

Ein einzelner Prompt:

```
claude-o-go -p "Explain this repository in one paragraph."
```

Modell fuer einen Lauf wechseln:

```
OPENCODE_GO_MODEL=minimax-m3 claude-o-go
```

4 Warum ein Proxy noetig ist

OpenCode Go stellt mehrere Modelle ueber eine Anthropic-kompatible Messages API bereit. Es ist eine Routing- und Abrechnungsschicht, kein Modell-Host. Jeder Request wird an die API des jeweiligen Modellanbieters weitergeleitet.

Beispiele:

```
qwen3.7-plus
minimax-m3
qwen3.7-max
```

Claude Code validiert Modellnamen lokal, bevor ein Request abgeschickt wird. Ein direkter Start mit einem OpenCode-Go-Modell wird deshalb abgelehnt:

```
ANTHROPIC_BASE_URL=https://opencode.ai/zen/go/v1 \
ANTHROPIC_API_KEY=... \
claude --model qwen3.7-plus
```

Der Proxy loest das, indem Claude Code lokal ein normales Claude-Alias wie **sonnet** sieht. Der Proxy schreibt den ausgehenden Request dann auf das OpenCode-Go-Modell um.

5 Exkurs: Localhost, Ports und Prozesse

Networking beginnt mit einer einfachen Frage: wohin soll ein Client verbinden?

- 127.0.0.1 bedeutet localhost. Die Adresse zeigt auf dieselbe Maschine.
- Ein Port waehlt einen konkreten lauschenden Prozess aus. Der Standard-Port ist 4141.
- Ein Server bindet sich an Adresse und Port. Der Proxy bindet sich an 127.0.0.1:4141.
- Ein Client verbindet sich zu Adresse und Port. Claude Code verbindet sich zur lokalen Proxy-URL:

```
http://127.0.0.1:4141
```

```
Claude Code Prozess
  verbindet zu 127.0.0.1:4141
```

```
Node Proxy Prozess
  lauscht auf 127.0.0.1:4141
  verbindet nach aussen zu opencode.ai:443
```

Der Proxy lauscht auf localhost, nicht auf 0.0.0.0. Das ist wichtig: 127.0.0.1 ist nur von deiner eigenen Maschine erreichbar. 0.0.0.0 wuerde den Listener auf allen Netzwerkinterfaces anbieten.

Nuetzliche Admin-Checks:

```
ss -ltnp | grep 4141
lsof -iTCP:4141 -sTCP:LISTEN
```

6 Exkurs: HTTP-Proxy vs Backend

Claude Code ist der Client. OpenCode Go ist das Upstream-Backend. Das lokale Node-Skript ist ein Proxy.

Ein Proxy nimmt einen Request von einer Seite an und sendet einen verwandten Request an eine andere Seite. Er kann den groessten Teil unveraendert lassen, einzelne Felder umschreiben und danach die Antwort zurueckreichen.

In diesem Projekt macht der Proxy drei wichtige Dinge:

1. Er nimmt Claude Codes Anthropic-Request unter `/v1/messages` an.
2. Er schreibt `body.model` von Claudes lokalem Alias auf das OpenCode-Go-Modell um.
3. Er leitet den Request an OpenCode Go weiter und streamt die Antwort zurueck.

Der Proxy ist kein Modell-Host. Er erzeugt keine Tokens. Er ist Glue-Code zwischen zwei APIs, die fast, aber nicht perfekt, kompatibel sind.

7 Modell-Kompatibilitaet

Jeder Upstream-Anbieter implementiert seinen Anthropic-kompatiblen Endpoint selbst. Deshalb variiert die Kompatibilitaet. Claude Code sendet einen vollstaendigen Anthropic-Payload mit Tools, Cache Control, Tool Choice und Anthropic-Beta-Headern.

Modell-ID	Anbieter	Funktioniert mit Claude Code Har-ness?
qwen3.7-plus	Alibaba (Qwen)	Ja
minimax-m3	MiniMax	Ja
glm-5.2	Z.ai (Zhipu AI)	Nein; invalid API parameter
glm-5.1	Z.ai (Zhipu AI)	Nein; wie glm-5.2
kimi-k2.7-code	Moonshot AI	Nein; lehnt Tool Function Names ab
deepseek-v4-pro	DeepSeek	Nein; erwartet OpenAI Tool Shape

Modelle desselben Anbieters verhalten sich meist wie das getestete Modell. Ungepruefte Modelle sollten vor produktiver Nutzung getestet werden.

Praktische Empfehlung:

```
export OPENCODE_GO_MODEL="qwen3.7-plus"    # oder minimax-m3
```

8 Request Flow

Wenn du Folgendes startest:

```
./claude-o-go -p "Reply with exactly: OK"
```

passiert Folgendes:

1. `claude-o-go` liest den OpenCode Go API Key aus `.env`.
2. Es startet den lokalen Proxy auf `http://127.0.0.1:4141`.
3. Es setzt `ANTHROPIC_BASE_URL=http://127.0.0.1:4141`.
4. Es startet Claude Code mit `claude -bare -model sonnet`.
5. Claude Code sendet einen Request an `/v1/messages`.
6. Der Proxy aendert das Modell im Request Body auf das OpenCode-Go-Modell.
7. Der Proxy leitet an `https://opencode.ai/zen/go/v1/messages` weiter.
8. OpenCode Go liefert die Modellantwort.
9. Der Proxy streamt die Antwort zurueck an Claude Code.

9 Exkurs: Streaming, SSE und Backpressure

LLM-Antworten werden normalerweise gestreamt. Das Backend wartet nicht, bis die komplette Antwort fertig ist, sondern sendet kleine Chunks, sobald sie verfuegbar sind.

Ein haeufiges Web-Muster dafuer ist SSE: Server-Sent Events. Ein SSE-Stream ist weiterhin HTTP, aber der Response Body bleibt offen und sendet ueber Zeit Events:

```
data: {"type":"content_block_delta","text":"hello"}
```

```
data: {"type":"content_block_delta","text":" world"}
```

```
data: [DONE]
```

Backpressure bedeutet, dass der Empfaenger Daten nicht immer so schnell verarbeiten kann, wie der Sender sie schreibt. In Node gibt `response.write(chunk) false` zurueck, wenn der Ausgabepuffer voll ist. Korrekte Proxy-Logik wartet dann auf `drain`, bevor sie weiterschreibt. Das verhindert Speicherwachstum und instabile lange Streams.

In diesem Projekt:

- Claude Code liest vom lokalen Proxy.
- Der Proxy liest von OpenCode Go.
- Der Proxy schreibt Chunks an Claude Code und beachtet Node-Stream-Backpressure.
- Wenn Claude Code die lokale Verbindung schliesst, bricht der Proxy den Upstream-Request ab.

Stream-Handling ist empfindlicher als ein einfacher JSON-Request. Ein normaler Request hat eine Antwort. Ein Stream ist ein laufendes Gespraech zwischen Sockets.

10 Exkurs: Haeufige Netzwerkfehler

Diese Fehlernamen gehoeren zum praktischen Admin-Wortschatz:

- **ECONNREFUSED**: niemand hat die Verbindung angenommen. Typische Ursachen: falscher Port, Proxy nicht gestartet oder Upstream nicht erreichbar.
- **ECONNRESET**: die Gegenseite hat eine bestehende Verbindung zurueckgesetzt. Typische Ursache: der Upstream hat den Socket mitten im Stream geschlossen.
- **UND_ERR_SOCKET**: Nodes Undici-Client sah einen Socket-Fehler. Meist Reset oder kaputter Stream.
- **ETIMEDOUT**: Verbindung oder Antwort dauerte zu lange. Ursache: Netzwerkpfad oder Backend ist zu langsam.
- **AbortError**: wir haben den Request absichtlich abgebrochen. Typische Ursache: Claude Code hat die lokale Verbindung geschlossen.

Praktische Regel: Retry vor dem Senden der Response-Header; nach Streaming-Beginn den Stream sauber mit einem Error-Event beenden. Sobald Header und Teillinhalt gesendet wurden, kann der Proxy daraus keine normale JSON 502-Antwort mehr machen.

11 Wichtige Dateien

11.1 claude-o-go

Der Launcher exportiert API Key, Modell und lokale Base URL und startet dann Claude Code:

```
export OPENCODE_GO_API_KEY="$api_key"
export OPENCODE_GO_MODEL="$model"
export ANTHROPIC_BASE_URL="http://127.0.0.1:${proxy_port}"
claude --bare --model "$claude_model_alias" "$@"
```

Wichtige Defaults und Umgebungsvariablen:

- **OPENCODE_GO_MODEL**: erforderlich; kein hartcodierter Launcher-Default.
- **OPENCODE_GO_PROXY_PORT**=4141.
- **OPENCODE_GO_UPSTREAM_RETRIES**=2.
- **CLAUDE_CODE_MODEL_ALIAS**=sonnet.
- **CLAUDE_CODE_OPENCODE_GO_BARE**=1.

11.2 opencode-go-claude-proxy.mjs

Der zentrale Rewrite im Proxy:

```
body.model = model
```

Bei Generierungsrequests streamt der Proxy die Antwort von OpenCode Go zurueck an Claude Code. Er beachtet Node-Stream-Backpressure, bricht den Upstream-Request ab, wenn Claude Code die lokale Verbindung schliesst, wiederholt retrybare Upstream-Verbindungsfehler vor dem Senden der Response-Header und wandelt Upstream-Resets mitten im Stream in ein sauberes Streaming-Error-Event um, statt den Proxy-Prozess zu beenden.

Der Proxy implementiert ausserdem Endpunkte, die Claude Code abfragt:

```
HEAD /  
HEAD /v1  
GET /v1/models  
POST /v1/messages/count_tokens  
POST /v1/messages
```

12 Lokale Checks

Vor dem Push ausfuehren:

```
npm run check
```

Das prueft die Syntax beider JavaScript-Einstiegspunkte.

13 Verifikation

Starte:

```
OPENCODE_GO_PROXY_LOG=1 ./claude-o-go -p "Reply with exactly: OK"
```

Proxy-Request-Logs sind im normalen Betrieb ausgeblendet, damit Claude Codes TUI sauber bleibt. Mit OPENCODE_GO_PROXY_LOG=1 achte auf Zeilen wie:

```
OpenCode Go Claude proxy listening on http://127.0.0.1:4141/v1 -> qwen3.7-plus  
POST /v1/messages  
POST /messages -> 200 as qwen3.7-plus  
OK
```

Die wichtigste Zeile ist:

```
POST /messages -> 200 as qwen3.7-plus
```

Das bedeutet: Claude Code hat an den lokalen Proxy gesendet, der Proxy hat an OpenCode Go weitergeleitet, OpenCode Go hat HTTP 200 geliefert, und das Log zeigt das verwendete Upstream-Modell.

14 Häufige Probleme

14.1 Proxy-Logs erscheinen in Claude Codes TUI

Normale Request-Logs sind standardmaessig ausgeblendet. Zum Debuggen:

```
OPENCODE_GO_PROXY_LOG=1 claude-o-go
```

Im normalen Betrieb unset lassen, damit die TUI sauber bleibt.

14.2 Selected model issue

Nutze den Launcher:

```
./claude-o-go
```

Rufe `claude -model qwen3.7-plus` nicht direkt auf.

14.3 Doppeltes /v1/v1/messages

ANTHROPIC_BASE_URL muss auf die Proxy-Wurzel zeigen:

```
http://127.0.0.1:4141
```

Sie darf nicht /v1 enthalten, weil Claude Code selbst /v1/messages anhaengt.

14.4 Claude Code zeigt sonnet

Das ist erwartet. Claude Code sieht `sonnet`; OpenCode Go erhaelt `qwen3.7-plus`. Das Proxy-Log ist die Quelle der Wahrheit.

15 Mental Model

Claude Code CLI

```
denkt model = sonnet
sendet Anthropic Request
|
v
```

Lokaler Proxy

```
schreibt model = qwen3.7-plus
leitet weiter
|
v
```

OpenCode Go

```
fuehrt das echte Modell aus
```

```
gibt Antwort zurueck
|
v
Claude Code CLI
zeigt die Antwort an
```

Die Claude-Code-Nutzung bleibt gleich, aber das Modell-Backend ist OpenCode Go.